

The Machine Basics

Youngjae Kim (Ph.D.)

Sogang University

Linux 운영체제 및 응용 (GITF315-01)

Machine Programming : Basics

- History of Intel processors and architectures
- Assembly Basics: Registers, operands, move
- Arithmetic & logical operations
- C, assembly, machine code

Intel x86 Processors

■ Dominate laptop/desktop/server market

- ARM architecture is a promising newcomer

■ Evolutionary design

- Backwards compatible up until 8086, introduced in 1978
- Added more features as time goes on
 - Now 3 volumes, about 5,000 pages of documentation

■ Complex instruction set computer (CISC)

- Many different instructions with many different formats
 - But, only small subset encountered with Linux programs
- Hard to match performance of Reduced Instruction Set Computers (RISC)
- But, Intel has done just that!
 - In terms of speed. Less so for low power.

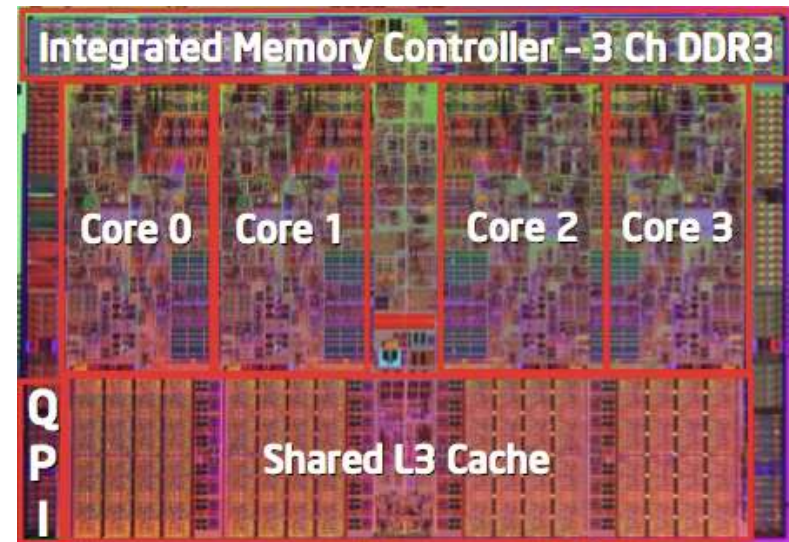
Intel x86 Evolution: Milestones

<i>Name</i>	<i>Date</i>	<i>Transistors</i>	<i>MHz</i>
■ 8086	1978	29K	5-10
▪ First 16-bit Intel processor. Basis for IBM PC & DOS ▪ 1MB address space			
■ 386	1985	275K	16-33
▪ First 32 bit Intel processor , referred to as IA32 ▪ Added “flat addressing”, capable of running Unix			
■ Pentium 4E	2004	125M	2800-3800
▪ First 64-bit Intel x86 processor, referred to as x86-64			
■ Core 2	2006	291M	1060-3333
▪ First multi-core Intel processor			
■ Core i7	2008	731M	1600-4400
▪ Four cores (our shark machines)			

Intel x86 Processors, cont.

■ Machine Evolution

■ 386	1985	0.3M
■ Pentium	1993	3.1M
■ Pentium/MMX	1997	4.5M
■ PentiumPro	1995	6.5M
■ Pentium III	1999	8.2M
■ Pentium 4	2000	42M
■ Core 2 Duo	2006	291M
■ Core i7	2008	731M
■ Core i7 Skylake	2015	1.9B



■ Added Features

- Instructions to support multimedia operations
- Instructions to enable more efficient conditional operations
- Transition from 32 bits to 64 bits
- More cores

Intel x86 Processors, cont.

■ Past Generations

Process technology

■ 1 st Pentium Pro	1995	600 nm
■ 1 st Pentium III	1999	250 nm
■ 1 st Pentium 4	2000	180 nm
■ 1 st Core 2 Duo	2006	65 nm

■ Recent & Upcoming Generations

1.	Nehalem	2008	45 nm
2.	Sandy Bridge	2011	32 nm
3.	Ivy Bridge	2012	22 nm
4.	Haswell	2013	22 nm
5.	Broadwell	2014	14 nm
6.	Skylake	2015	14 nm
7.	Kaby Lake	2016	14 nm
8.	Coffee Lake	2017	14 nm
9.	Cannon Lake	2018	10 nm
10.	Ice Lake	2019	10 nm
11.	Tiger Lake	2020	10 nm
12.	Alder Lake	2022	“intel 7” (10nm+++)

Process technology dimension
= width of narrowest wires
(10 nm ≈ 100 atoms wide)

(But this is changing now.)

Machine Programming : Basics

- History of Intel processors and architectures
- **Assembly Basics: Registers, operands, move**
- Arithmetic & logical operations
- C, assembly, machine code

How Does a Computer Understand Code?

- Short answer: Bits!
- Not the complete picture, let's take it bit by bit



How Does a Person Understand English?

■ Let's see how humans understand English

- English uses an alphabet

- A, B, C ...

- Examine a sentence:

- Subject: (**I**) core noun of the sentence
 - Object: (**15213**) supporting noun
 - Verbs: (**love**) actions associating subject and object

I love 15213!

How Does a CPU Understand Code (cont)?

■ What's the language of the CPU?

- Basic building block is bits
- A sentence in English is like a program
- Subject and Object?
 - Data Types: Integers, Floats, ...
- Verb?
 - Arithmetic Operations: +, -, /, *, <<, >>, ...

1 + 2

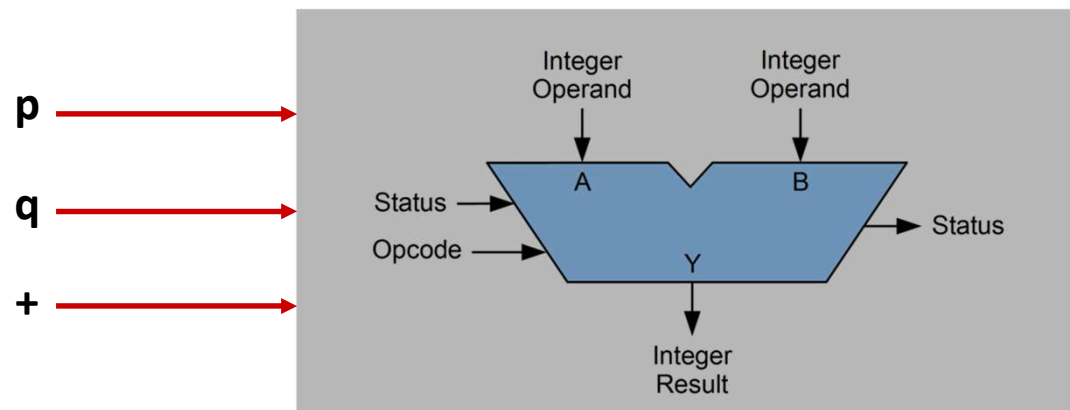
■ CPU: Take data, apply action, use result

- Let's encode: 1 + 2
 - (+, 1, 2)

Generalize Our Encoding

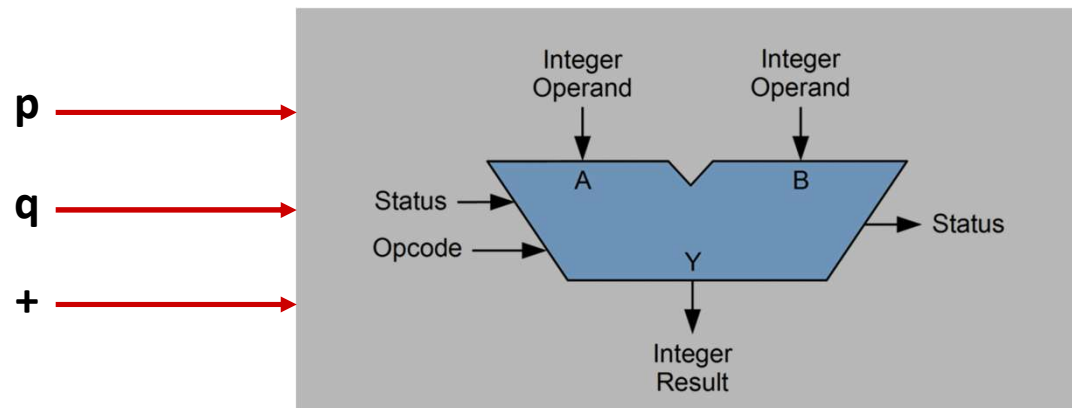
- $1 + 2 \rightarrow (+, 1, 2)$
- $3 + 4 \rightarrow (+, 3, 4)$
- Let's make specialized hardware in our CPU for $+$, $-$, ...

1 + 2



- **Separate operations from data**
 - $p = 1$
 - $q = 2$
 - $(+, p, q)$

Generalize Our Data Pipeline



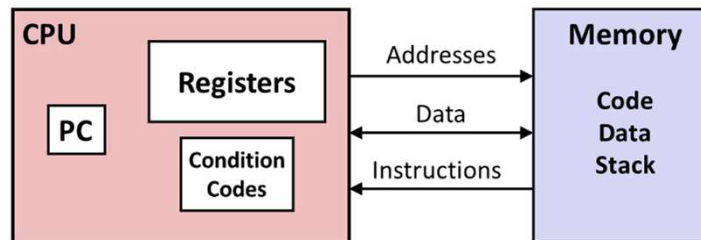
- CPU needs to supply p , q to our arithmetic unit
- CPU uses registers to store information for the ALU
 - $(+, \%p, \%q)$
- But where do the registers get information?
 - From the program, and in memory
 - $(\text{Load}, 0x1, \%p)$
 - $(\text{Load}, \&0x7FFFF7AFCBA0, \%q)$

Levels of Abstraction

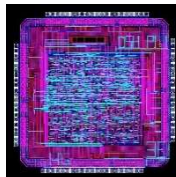
C programmer

```
#include <stdio.h>
int main() {
    int i, n = 10, t1 = 0, t2 = 1, nxt;
    for (i = 1; i <= n; ++i) {
        printf("%d, ", t1);
        nxt = t1 + t2;
        t1 = t2;
        t2 = nxt; }
    return 0; }
```

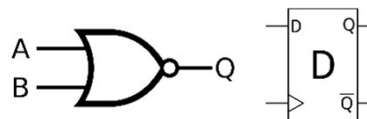
Assembly programmer



Computer Designer



Gates, clocks, circuit layout, ...

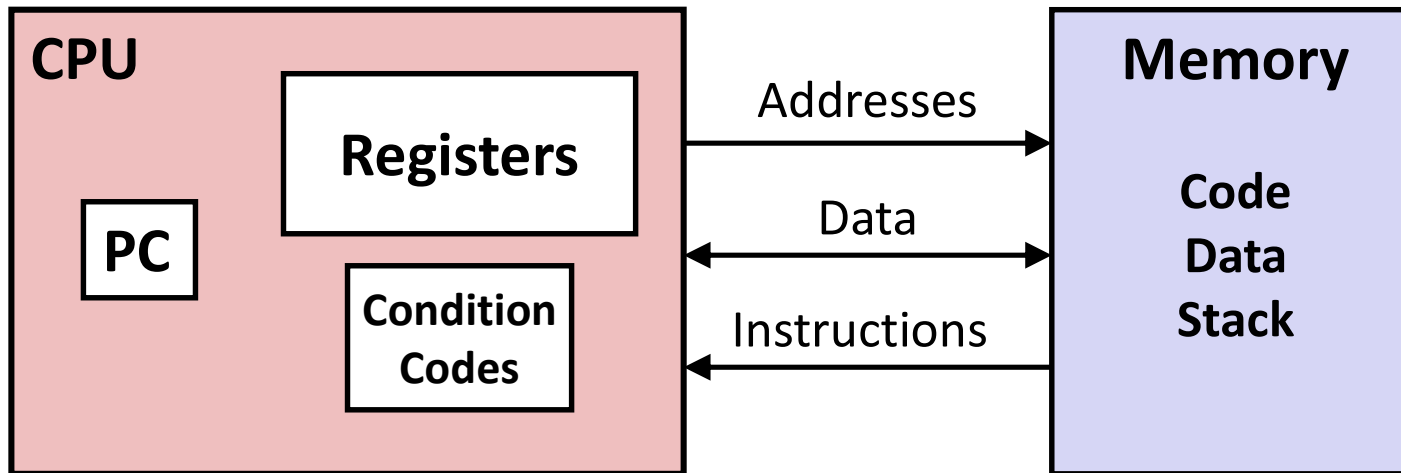


Definitions

■ Code Forms:

- **Machine Code:** The byte-level programs that a processor executes
- **Assembly Code:** A text representation of machine code
 - `mov al, 61h`

Assembly/Machine Code View



Programmer-Visible State

■ PC: Program counter

- Address of next instruction
- Called “RIP” (x86-64)

■ Register file

- Heavily used program data

■ Condition codes

- Store status information about most recent arithmetic or logical operation
- Used for conditional branching



■ Memory

- Byte addressable array
- Code and user data
- Stack to support procedures

x86-64 Integer Registers

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- Can reference low-order 4 bytes (also low-order 1 & 2 bytes)
- Not part of memory (or cache)

Some History: IA32 Registers

general purpose	%eax	%ax	%ah	%al	Origin (mostly obsolete) <i>accumulate</i>
	%ecx	%cx	%ch	%cl	<i>counter</i>
	%edx	%dx	%dh	%dl	<i>data</i>
	%ebx	%bx	%bh	%bl	<i>base</i>
	%esi	%si			<i>source index</i>
	%edi	%di			<i>destination index</i>
	%esp	%sp			<i>stack pointer</i>
	%ebp	%bp			<i>base pointer</i>

16-bit virtual registers
(backwards compatibility)

Assembly Characteristics: Operations

■ Data Transfer Instructions

- Move data between **memory** and **registers** — the fundamental storage locations inside the CPU.
- **Load:**
 - Transfer data from memory to a register.
→ Example: MOV R1, [1000]
 - Loads the value from memory address 1000 into register R1.
- **Store:**
 - Transfer data from a register to memory.
→ Example: MOV [1000], R1
 - Stores the contents of R1 into memory address 1000.
- **Register-to-register moves:**
 - Sometimes data is moved between registers without involving memory.
→ Example: MOV R2, R1

Assembly Characteristics: Operations

■ Arithmetic and Logical Instructions

- Perform mathematical or logical operations on register or memory values.
- Examples:
 - `ADD R1, R2` $\rightarrow R1 = R1 + R2$
 - `SUB R1, R2` $\rightarrow R1 = R1 - R2$
 - `AND R1, R2` $\rightarrow$ Bitwise AND operation
 - `CMP R1, R2` $\rightarrow$ Compare two registers (used for conditional branching)

Assembly Characteristics: Operations

■ Control Transfer Instructions

- **Change the sequence of execution** — i.e., they tell the CPU *what instruction to execute next*.
- **Unconditional jumps:**
 - Always jump to a new instruction or procedure.
→ Example: `JMP target_label`
Execution continues from `target_label`.
- **Conditional branches:**
 - Jump only if a specific condition is true (e.g., result of comparison).
→ Example: `JE target_label` (jump if equal),
`JNE target_label` (jump if not equal)
- **Indirect branches:**
 - Jump to the address stored in a register or memory location.
→ Example: `JMP [R1]`
(The jump target is determined dynamically.)

Moving Data

■ Moving Data

`movq Source, Dest`

■ Operand Types

- **Immediate:** Constant integer data
 - Example: `$0x400`, `$-533`
 - Like C constant, but prefixed with `'$'`
 - Encoded with 1, 2, or 4 bytes
- **Register:** One of 16 integer registers
 - Example: `%rax`, `%r13`
 - But `%rsp` (stack pointer) reserved for special use
 - Others have special uses for particular instructions
- **Memory** 8 consecutive bytes of memory at address given by register
 - Simplest example: `(%rax)`
 - Various other “addressing modes”

<code>%rax</code>
<code>%rcx</code>
<code>%rdx</code>
<code>%rbx</code>
<code>%rsi</code>
<code>%rdi</code>
<code>%rsp</code>
<code>%rbp</code>
<code>%rN</code>

movq Operand Combinations

	Source	Dest	Src, Dest	C Analog
movq	Imm	Reg	movq \$0x4, %rax	temp = 0x4;
		Mem	movq \$-147, (%rax)	*p = -147;
	Reg	Reg	movq %rax, %rdx	temp2 = temp1;
		Mem	movq %rax, (%rdx)	*p = temp;
	Mem	Reg	movq (%rax), %rdx	temp = *p;

Cannot do memory-memory transfer with a single instruction

Simple Memory Addressing Modes

■ Normal (R) Mem[Reg[R]]

- Register R specifies memory address
- Aha! Pointer dereferencing in C

```
movq (%rcx) , %rax
```

■ Displacement D(R) Mem[Reg[R]+D]

- Register R specifies start of memory region
- Constant displacement D specifies offset

```
movq 8(%rbp) , %rdx
```

Example of Simple Addressing Modes

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
swap:
    movq    (%rdi), %rax
    movq    (%rsi), %rdx
    movq    %rdx, (%rdi)
    movq    %rax, (%rsi)
    ret
```

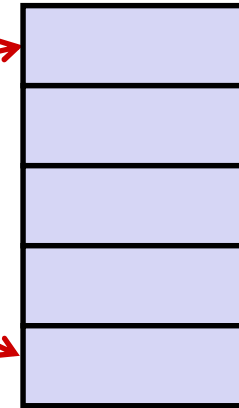

Understanding Swap()

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

Registers

%rdi	
%rsi	
%rax	
%rdx	

Memory



Register	Value
%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

swap:

movq (%rdi), %rax # t0 = *xp

movq (%rsi), %rdx # t1 = *yp

movq %rdx, (%rdi) # *xp = t1

movq %rax, (%rsi) # *yp = t0

ret

Understanding Swap()

Registers

<code>%rdi</code>	<code>0x120</code>
<code>%rsi</code>	<code>0x100</code>
<code>%rax</code>	
<code>%rdx</code>	

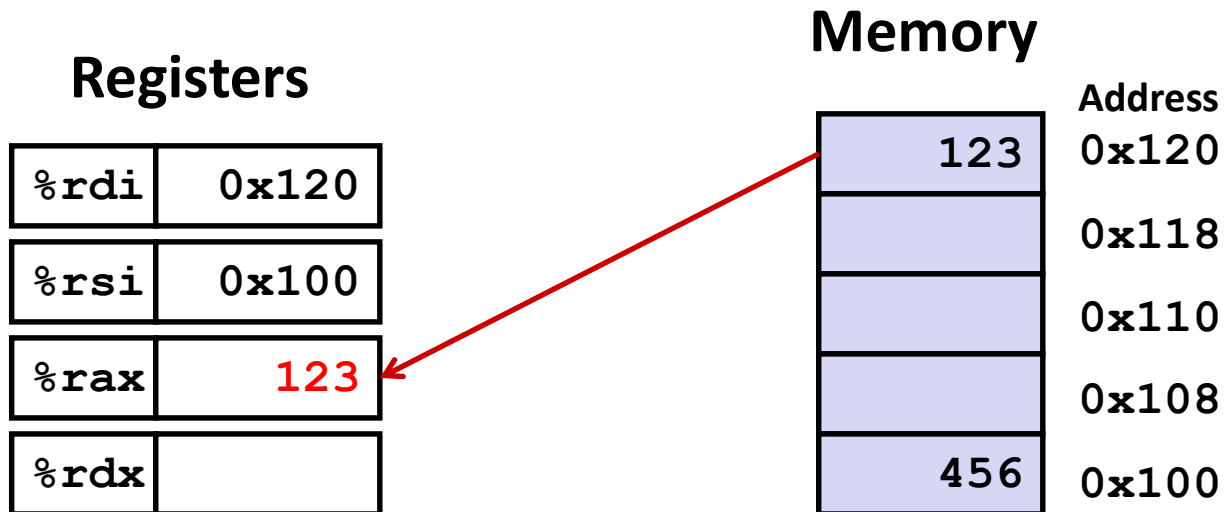
Memory

Address
<code>0x120</code>
<code>0x118</code>
<code>0x110</code>
<code>0x108</code>
<code>0x100</code>

swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

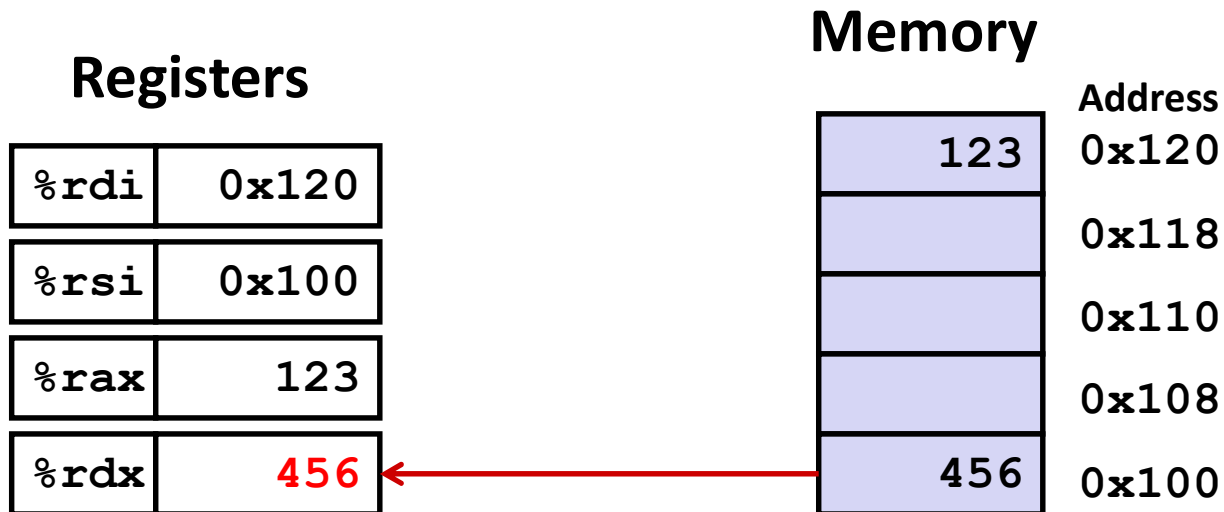
Understanding Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

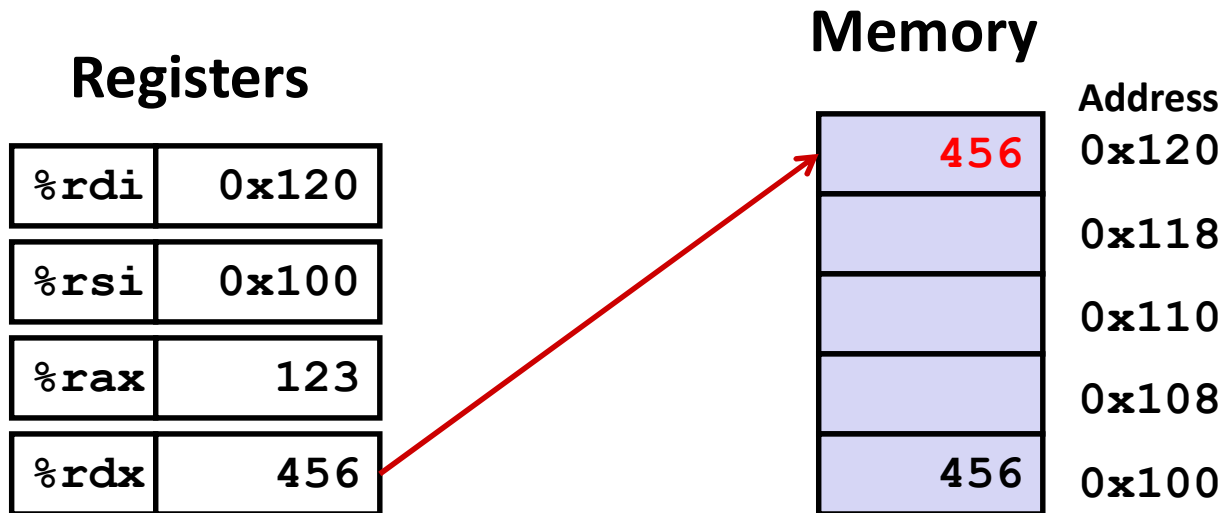
Understanding Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

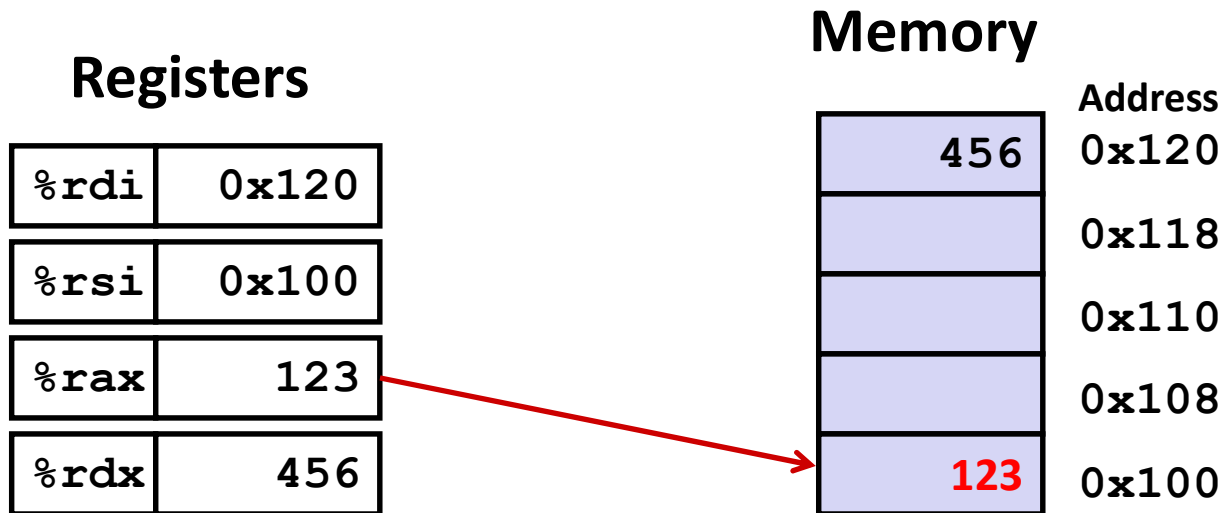
Understanding Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Understanding Swap()



swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Simple Memory Addressing Modes

■ Normal (R) Mem[Reg[R]]

- Register R specifies memory address
- Aha! Pointer dereferencing in C

```
movq (%rcx) , %rax
```

■ Displacement D(R) Mem[Reg[R]+D]

- Register R specifies start of memory region
- Constant displacement D specifies offset

```
movq 8(%rbp) , %rdx
```

Machine Programming : Basics

- History of Intel processors and architectures
- Assembly Basics: Registers, operands, move
- **Arithmetic & logical operations**
- C, assembly, machine code

Some Arithmetic Operations

■ Two Operand Instructions:

<i>Format</i>	<i>Computation</i>	
<code>addq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} + \text{Src}$
<code>subq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} - \text{Src}$
<code>imulq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} * \text{Src}$
<code>salq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} \ll \text{Src}$ <i>Also called shlq</i>
<code>sarq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} \gg \text{Src}$ <i>Arithmetic</i>
<code>shrq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} \gg \text{Src}$ <i>Logical</i>
<code>xorq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} \wedge \text{Src}$
<code>andq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} \& \text{Src}$
<code>orq</code>	<code>Src, Dest</code>	$\text{Dest} = \text{Dest} \text{Src}$

■ Watch out for argument order! *Src, Dest*

■ No distinction between signed and unsigned int (why?)

Some Arithmetic Operations

■ One Operand Instructions

<code>incq</code>	<i>Dest</i>	$Dest = Dest + 1$
<code>decq</code>	<i>Dest</i>	$Dest = Dest - 1$
<code>negq</code>	<i>Dest</i>	$Dest = -Dest$
<code>notq</code>	<i>Dest</i>	$Dest = \sim Dest$

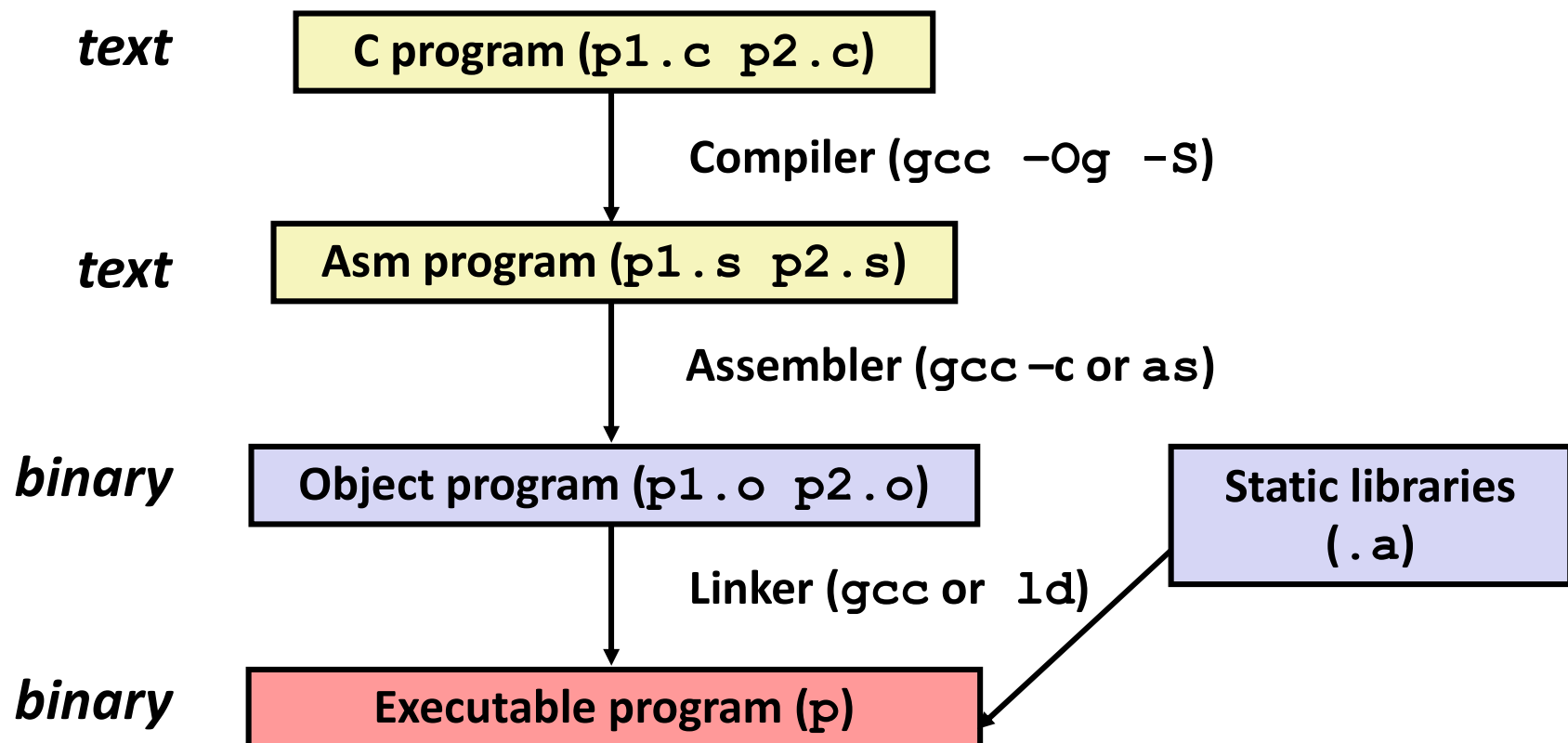
■ See book for more instructions

Machine Programming : Basics

- History of Intel processors and architectures
- Assembly Basics: Registers, operands, move
- Arithmetic & logical operations
- **C, assembly, machine code**

Turning C into Object Code

- Code in files `p1.c` `p2.c`
- Compile with command: `gcc -Og p1.c p2.c -o p`
 - Use debugging-friendly optimizations (`-Og`)
 - Put resulting binary in file `p`



Compiling Into Assembly

C Code (sum.c)

```
long plus(long x, long y);

void sumstore(long x, long y,
              long *dest)
{
    long t = plus(x, y);
    *dest = t;
}
```

Generated x86-64 Assembly

```
sumstore:
    pushq    %rbx
    movq     %rdx, %rbx
    call     plus
    movq     %rax, (%rbx)
    popq     %rbx
    ret
```

gcc -Og -S sum.c

Produces file sum.s

What it really looks like

```
        .globl  sumstore
        .type   sumstore, @function
sumstore:
.LFB35:
        .cfi_startproc
        pushq   %rbx
        .cfi_def_cfa_offset 16
        .cfi_offset 3, -16
        movq    %rdx, %rbx
        call    plus
        movq    %rax, (%rbx)
        popq    %rbx
        .cfi_def_cfa_offset 8
        ret
        .cfi_endproc
.LFE35:
        .size   sumstore, .-sumstore
```

What it really looks like

```
.globl sumstore
.type sumstore, @function
sumstore:
.LFB35:
.cfi_startproc
pushq %rbx
.cfi_def_cfa_offset 16
.cfi_offset 3, -16
movq %rdx, %rbx
call plus
movq %rax, (%rbx)
popq %rbx
.cfi_def_cfa_offset 8
ret
.cfi_endproc
.LFE35:
.size sumstore, .-sumstore
```

Things that look weird
and are preceded by a '
are generally directives.

```
sumstore:
    pushq    %rbx
    movq     %rdx, %rbx
    call     plus
    movq     %rax, (%rbx)
    popq     %rbx
    ret
```

Machine Programming : Summary

■ History of Intel processors and architectures

- Evolutionary design leads to many quirks and artifacts

■ C, assembly, machine code

- New forms of visible state: program counter, registers, ...
- Compiler must transform statements, expressions, procedures into low-level instruction sequences

■ Assembly Basics: Registers, operands, move

- The x86-64 move instructions cover wide range of data movement forms

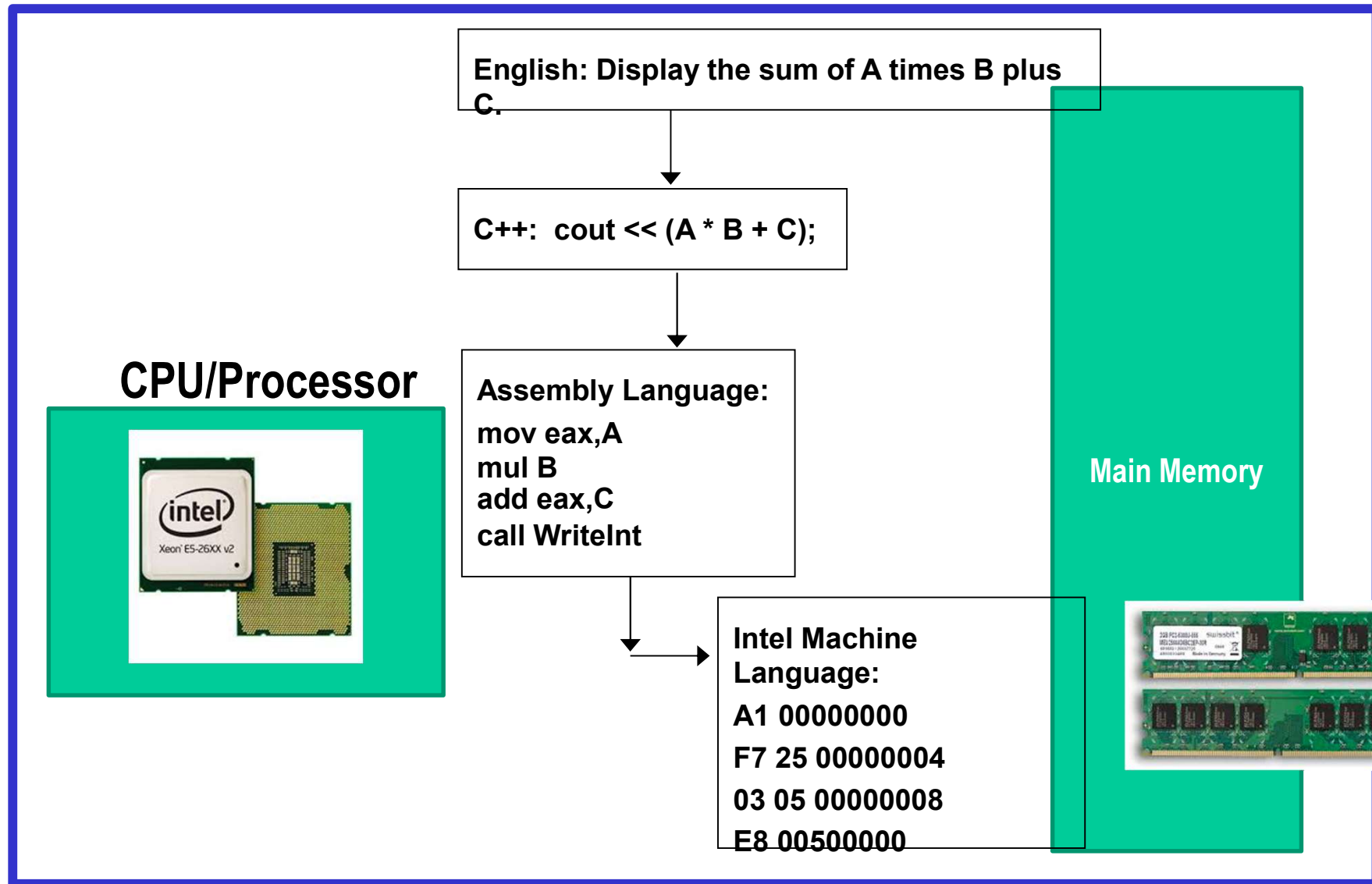
■ Arithmetic

- C compiler will figure out different instruction combinations to carry out computation

Computer Hardware and Software

- **Computer System Architecture 101**
- **Multiprocessor and Cache Coherence**
- **Instruction Pipelining**

Computer System Architecture 101

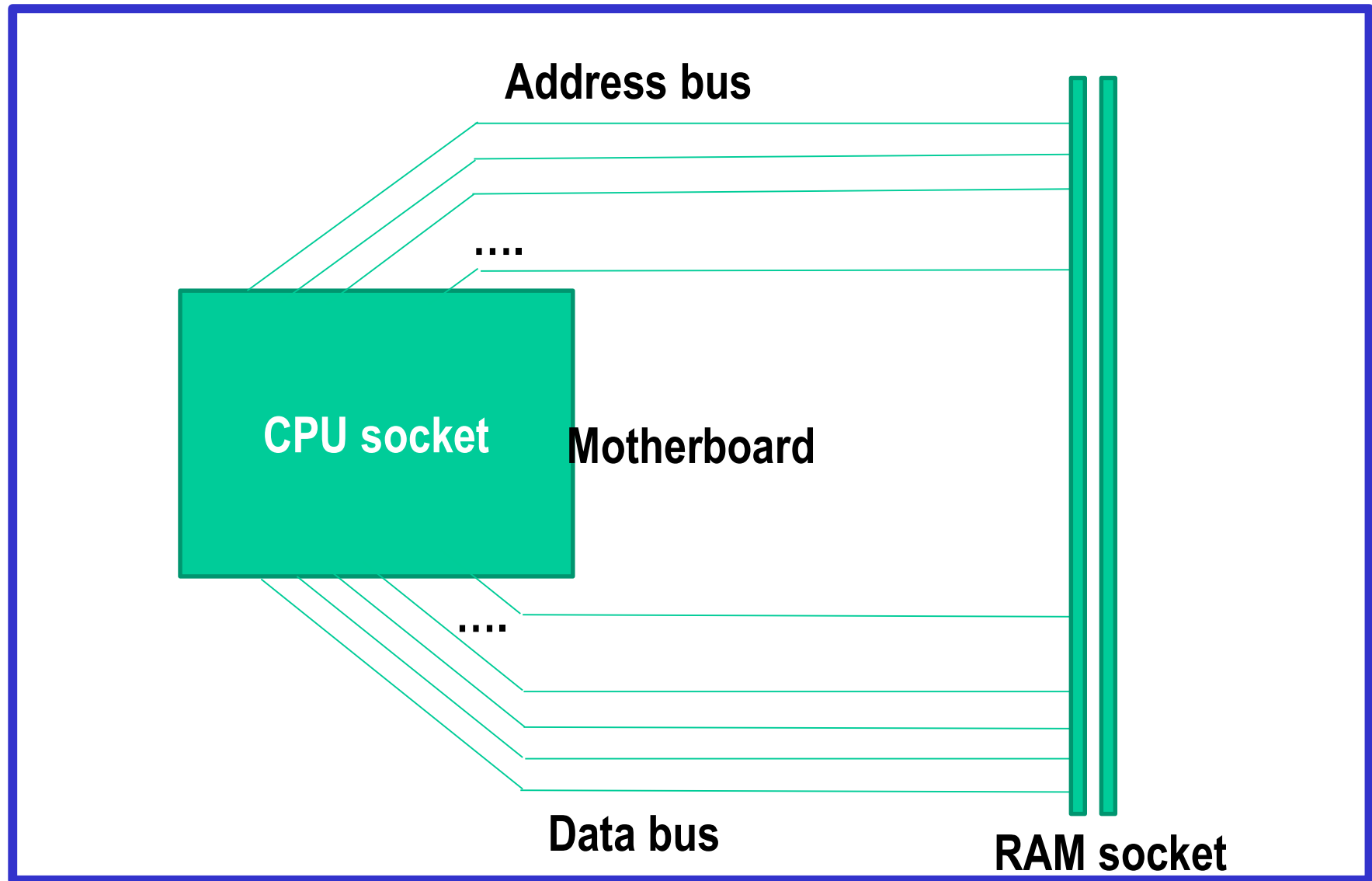


Computer System Architecture 101

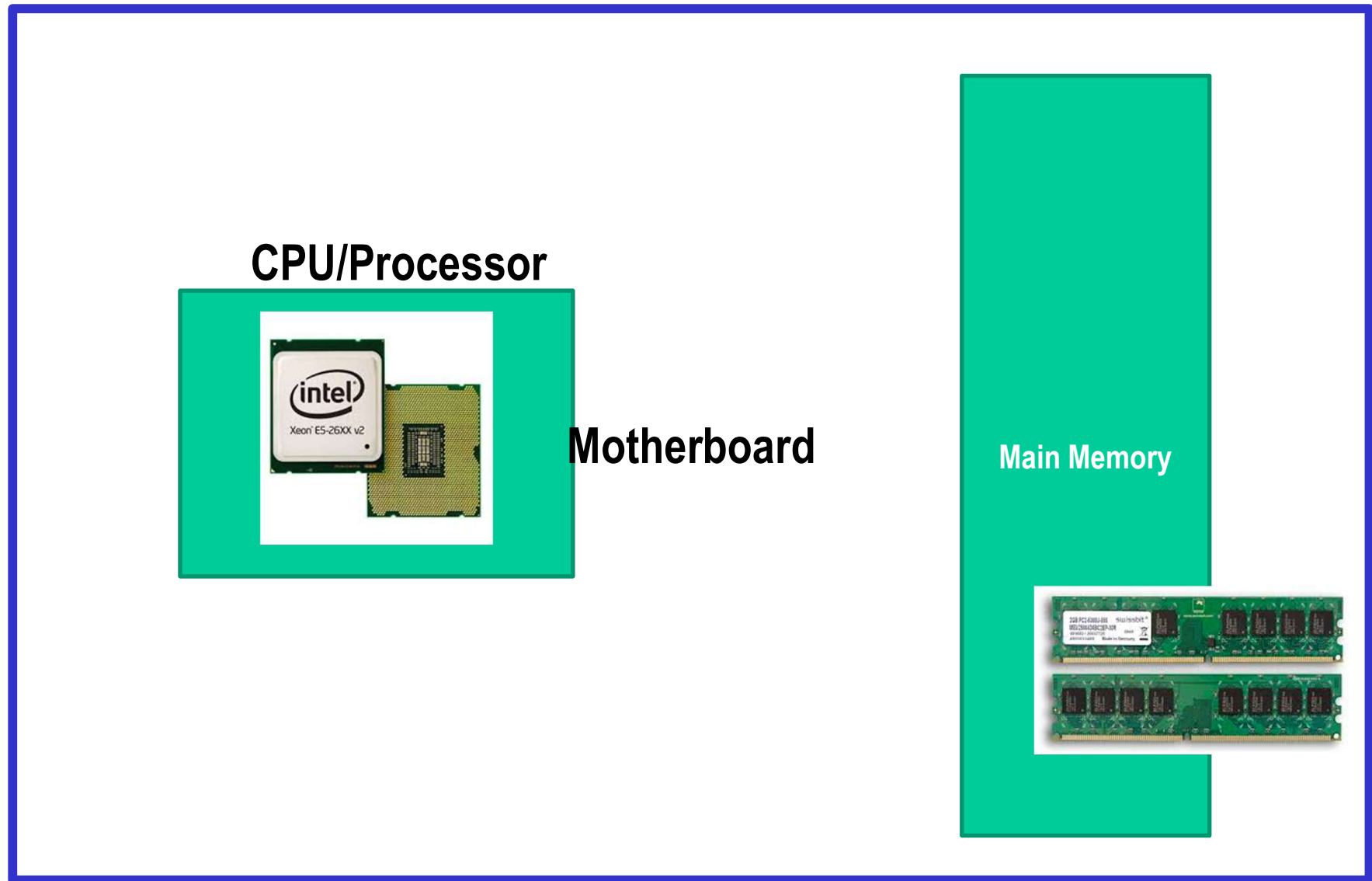


Motherboard

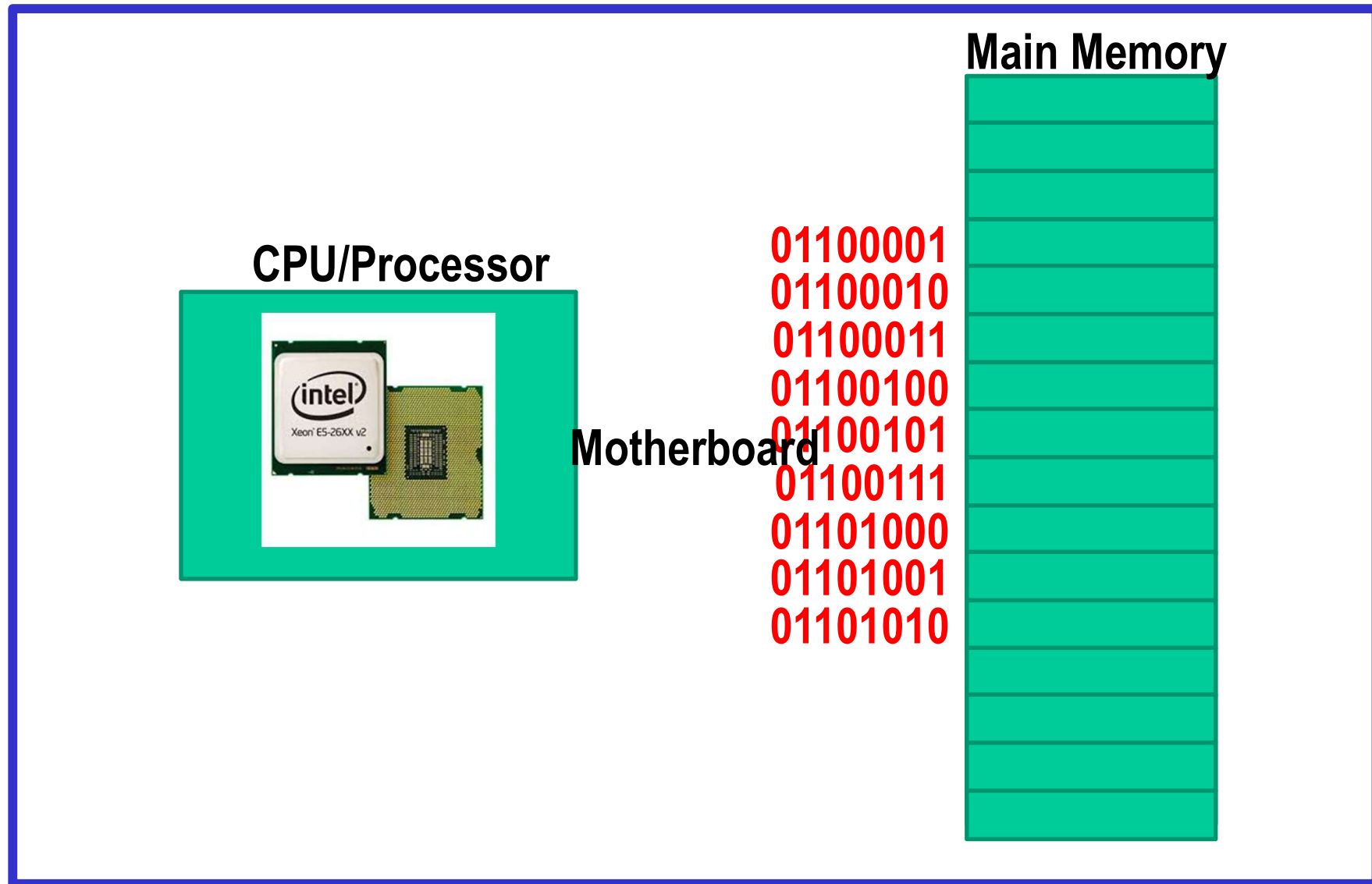
Computer System Architecture 101



Computer System Architecture 101



Computer System Architecture 101



Computer System Architecture 101

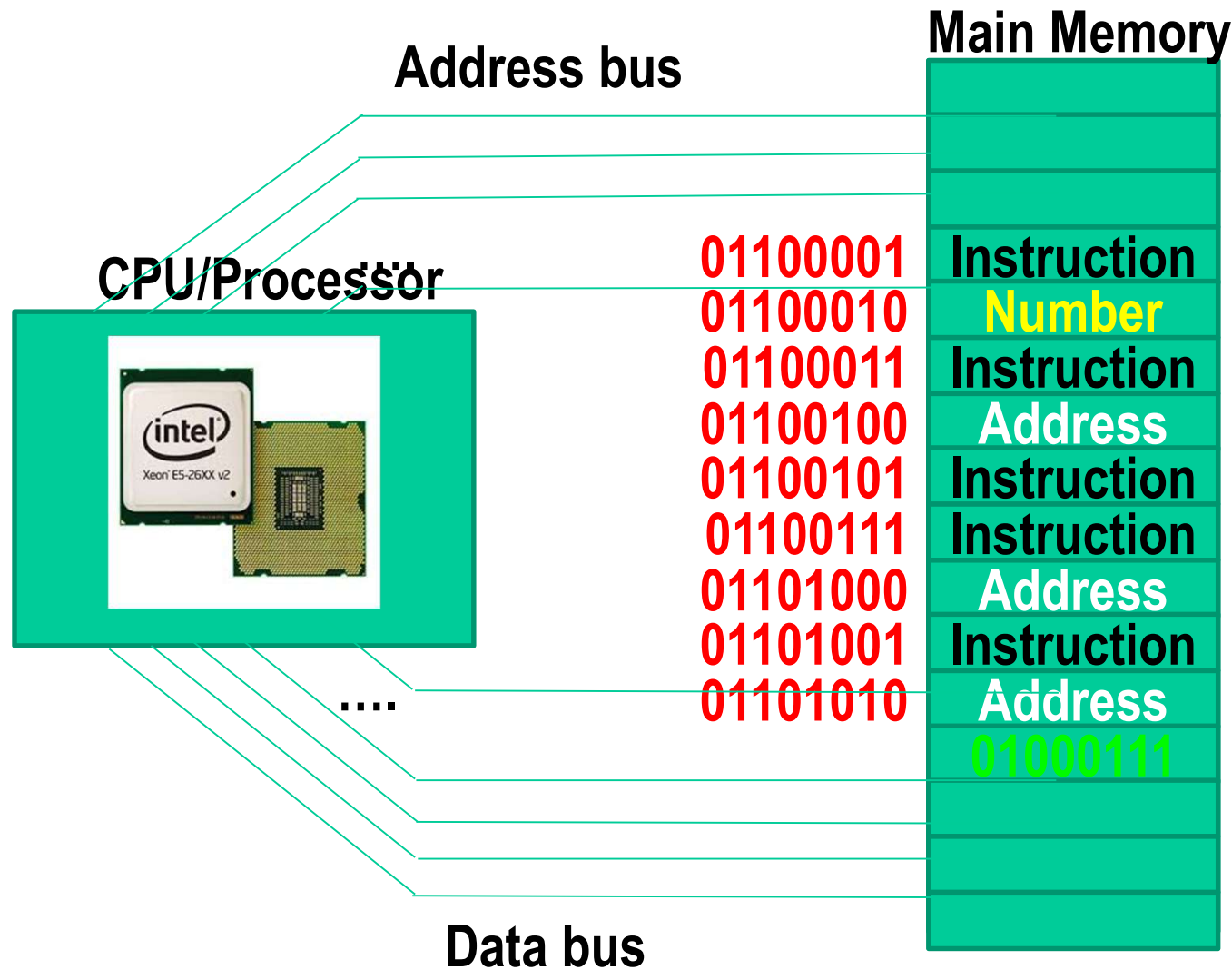
CPU/Processor



Main Memory

01100001	Instruction
01100010	Number
01100011	Instruction
01100100	Address
01100101	Instruction
01100111	Instruction
01101000	Address
01101001	Instruction
01101010	Address
01000111	

Computer System Architecture 101



Computer System Architecture 101

CPU/Processor



Instruction Set

LOAD a number from RAM into the CPU

STORE a number from the CPU bank
back out to RAM

ADD two numbers together

COMPARE one number with another

JUMP to another address in RAM

...

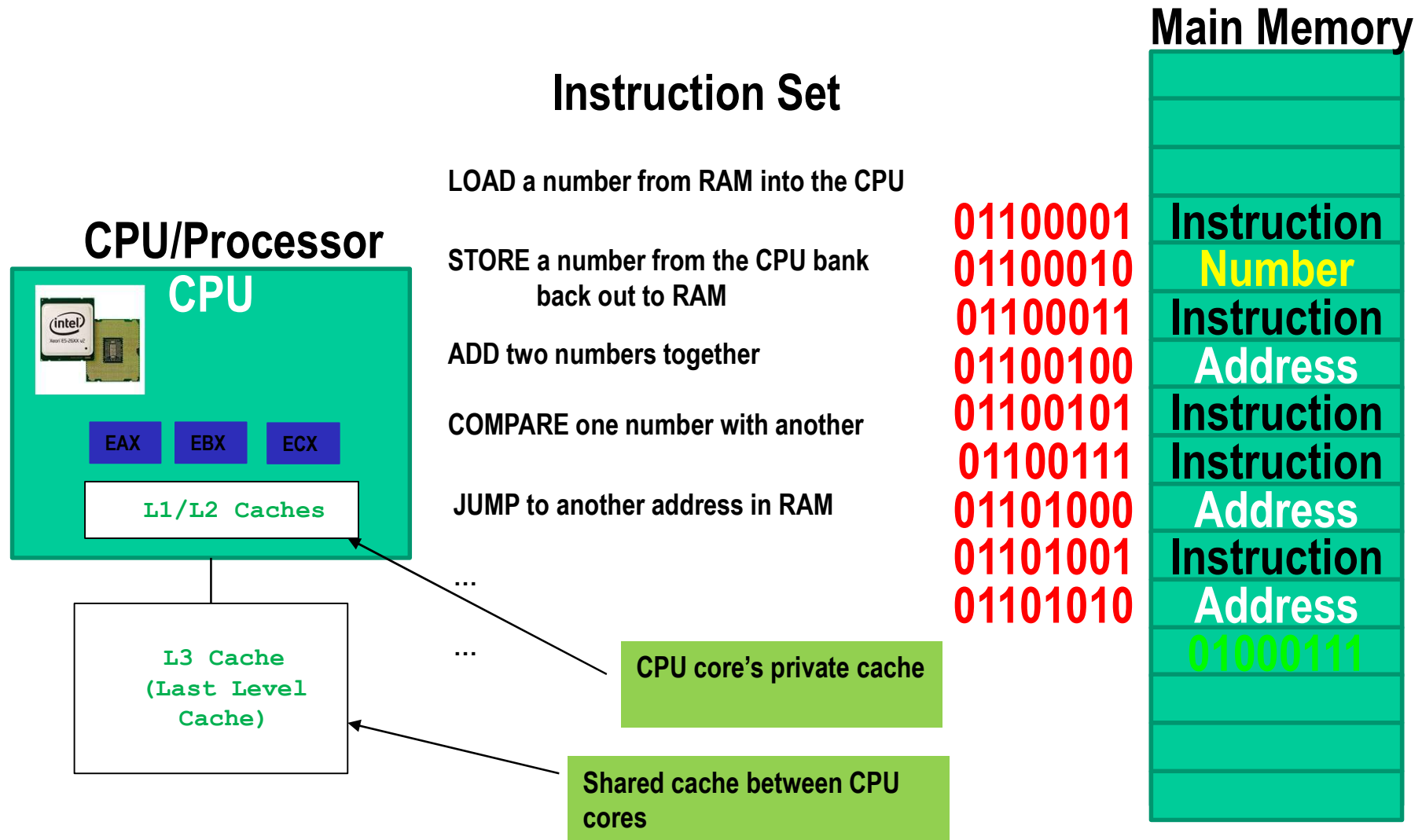
...

01100001
01100010
01100011
01100100
01100101
01100111
01101000
01101001
01101010

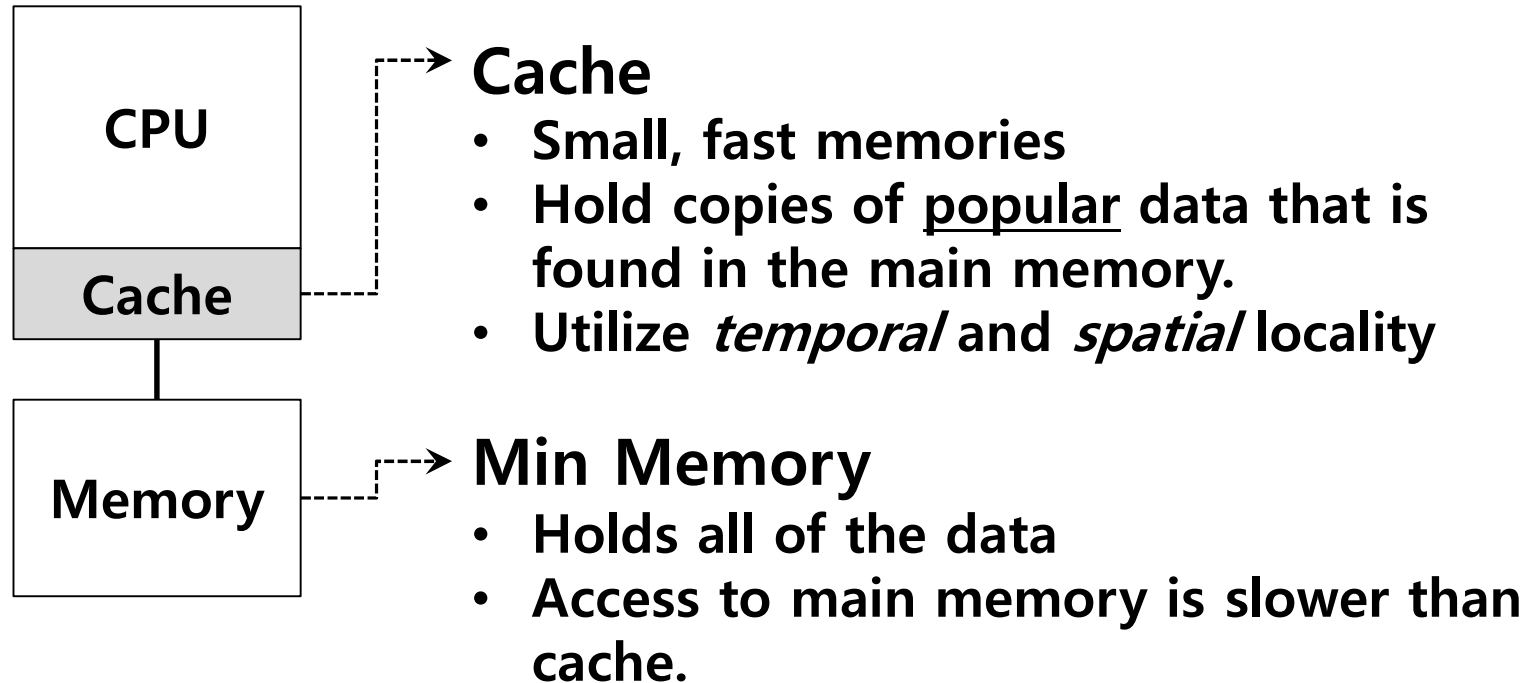
Main Memory

Instruction
Number
Instruction
Address
Instruction
Instruction
Address
Instruction
Address
01000111

Computer System Architecture 101



Single CPU with cache

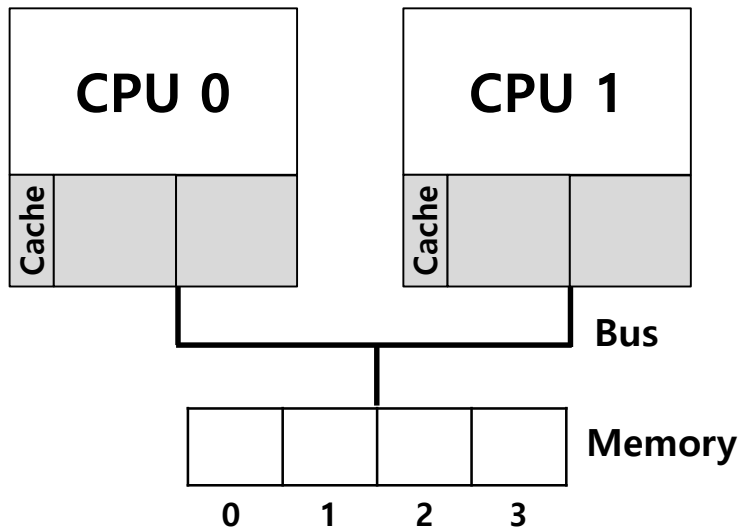


By keeping data in cache, the system can make slow memory
appear to be a fast one

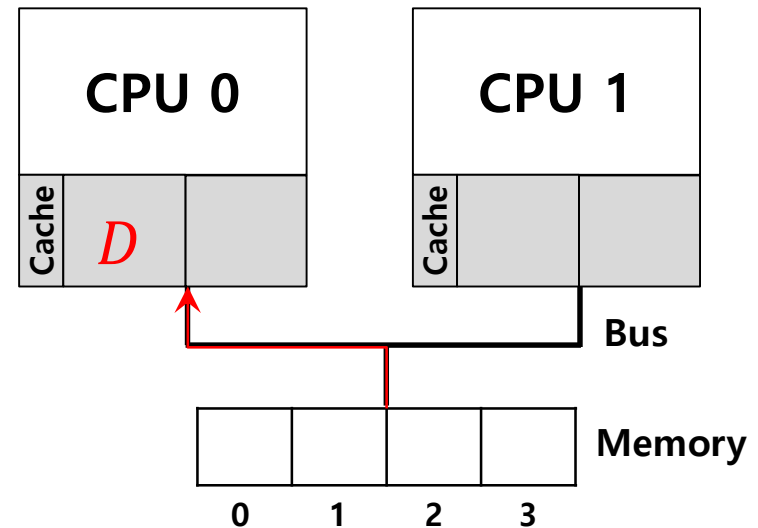
Cache coherence

- Consistency of shared resource data stored in multiple caches.

0. Two CPUs with caches sharing memory



1. CPU0 reads a data at address 1.

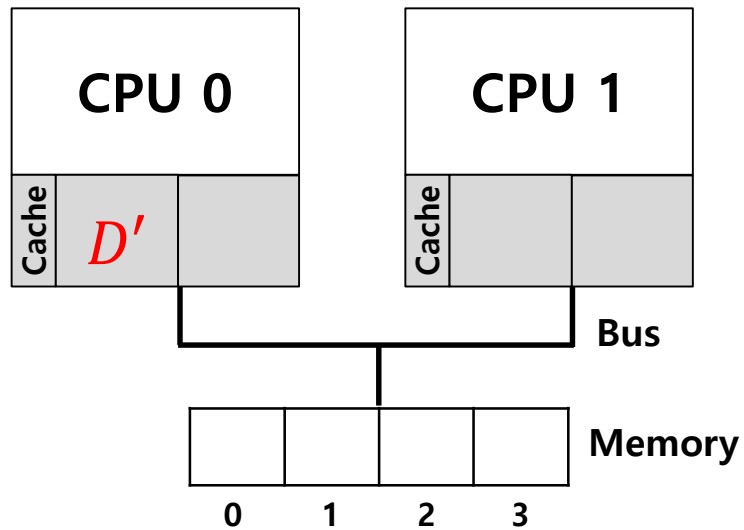


- **Bus snooping**

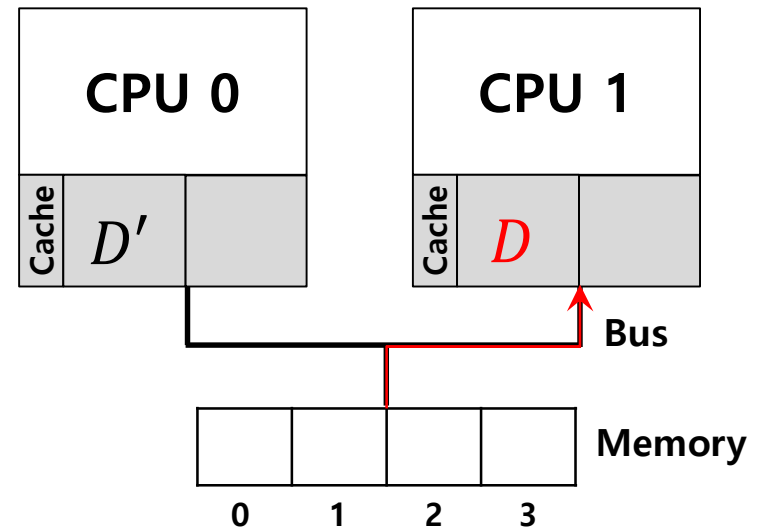
- Each cache pays attention to memory updates by **observing the bus**.
- When a CPU sees an update for a data item it holds in its cache, it will notice the change and either invalidate its copy or update it.

Cache coherence (Cont.)

2. D is updated and CPU1 is scheduled.



3. CPU1 re-reads the value at address A



CPU1 gets the **old value D** instead of the correct value D' .

Cache Affinity

- Keep a process on **the same CPU** if at all possible
 - A process builds up a fair bit of state in the cache of a CPU.
 - The next time the process run, it will run faster if some of its state is *already present* in the cache on that CPU.

A multiprocessor scheduler should consider **cache affinity when making its scheduling decision.**

Instruction Pipelining

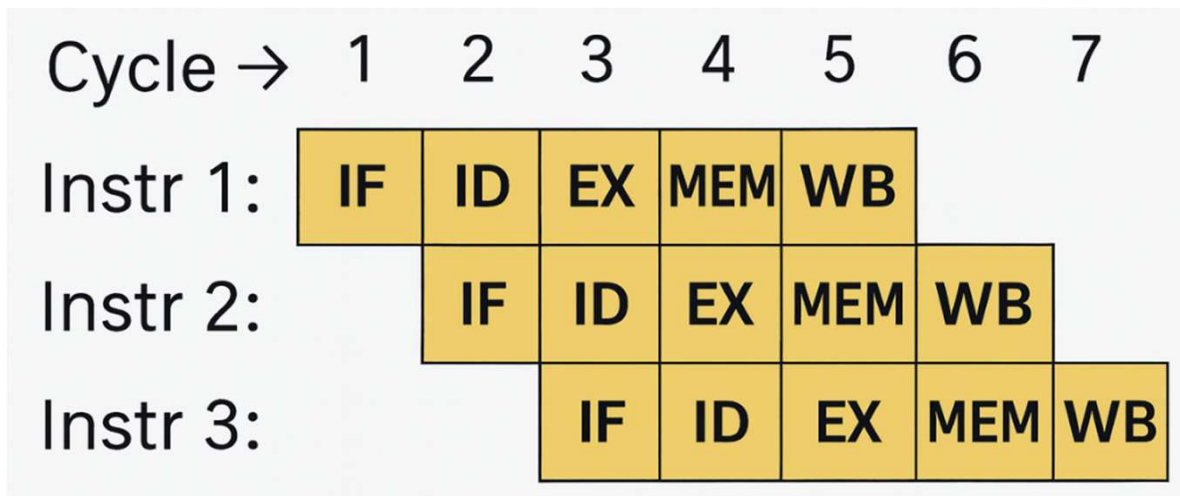
■ What is Instruction Pipelining?

- Technique used to increase instruction throughput
- Overlaps multiple instructions using pipeline stages
- Each stage works on a part of an instruction simultaneously
- Analogy: Like an assembly line — one instruction fetched, another decoded, another executed

■ Classic 5-Stage RISC Pipeline

- ① IF – Instruction Fetch: Fetch from memory
- ② ID – Instruction Decode / Register Fetch: Decode and read registers
- ③ EX – Execute: ALU operation or address calculation
- ④ MEM – Memory Access: Load or store data
- ⑤ WB – Write Back: Write result to register file

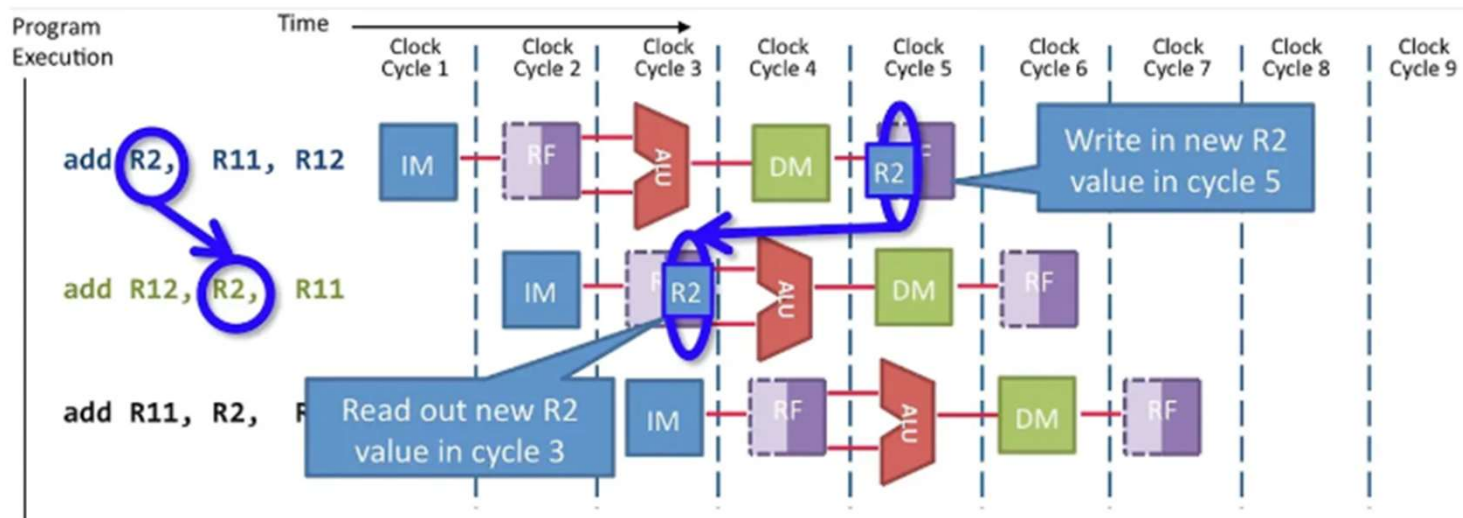
Example Timing Diagram



- Multiple instructions execute in parallel
- Throughput increases even though latency stays the same

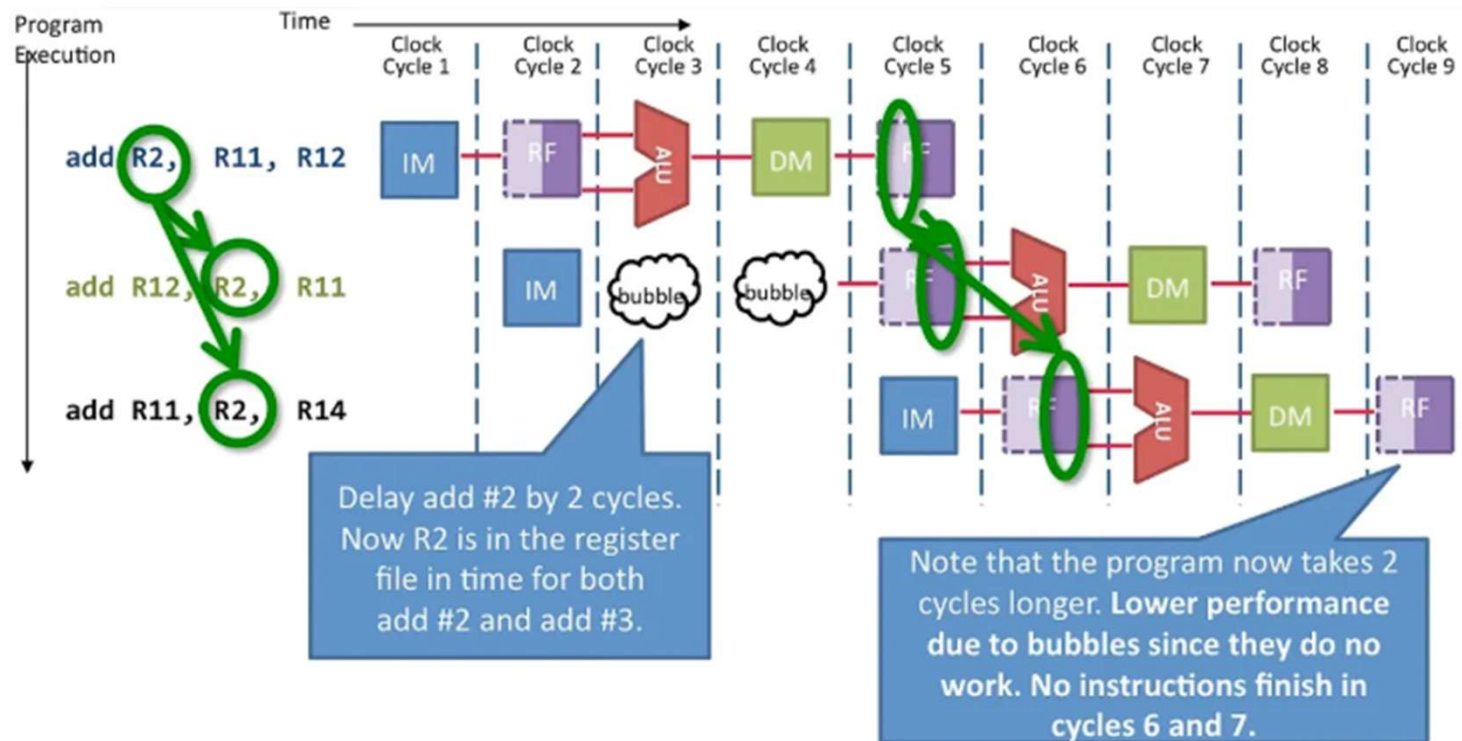
Pipeline Hazards

- **Structural Hazards:** Hardware resource conflicts
- **Data Hazards:** One instruction depends on another's result
 - Example: add r1,r2,r3 followed by sub r4,r1,r5
- **Example**



Pipeline Hazards

■ Solutions: Insert Bubbles, Out-of-Order Execution



Computer HW&SW : Summary

■ Computer System and Its Operation Procedures

- CPU (ALU, FPU, CU, Registers,..), Address Bus, Data Bus, Control Bus, ...

■ Multiprocessor Architecture

- Cache Coherence Protocols (Invalidate-, Update- Protocols)
- Cache affinity affects performance...

■ Instruction Pipelining and Pipelining Hazards

- Structural Hazards: Hardware resource conflicts